

Demystifying Data Structures: Arrays as Memory

CMPT 211: Implementing CLRS with Raw Python Lists

Bernal Manzanilla Saavedra

Concordia University of Edmonton

Winter 2026

The Illusion of Python's Magic

Python abstracts away the hardware. When you use a standard Python `list`, the interpreter does heavy lifting behind the scenes to dynamically resize memory and shift elements.

But true software engineers must understand the hardware limits. In Chapter 10 of *Introduction to Algorithms* (CLRS), dynamic structures are built using rigid memory allocations and raw pointers.

Our Rule for Today: We will simulate a computer's RAM using a single, fixed-size Python list. We are absolutely forbidden from using `.append()`, `.insert()`, or `.remove()`. We will build Stacks, Queues, Linked Lists, Trees, and Heaps using **ONLY** integer indices as our pointers!

1. The Stack: Array with a Single Boundary Pointer

A Stack (Last-In, First-Out) is the simplest dynamic set. We allocate a block of memory and use a single integer variable, `top`, to act as our pointer.

Time Complexity: $O(1)$ for Insert/Delete. **I/O Cost:** Extremely low (no shifting).

```
# CLRS Stack Translation (Using ConcordiaStudent objects)
memory_pool = [None] * 100
top_pointer = -1 # -1 indicates an empty stack

def push_student(student_obj):
    global top_pointer
    # Move the pointer UP to the next available memory slot
    top_pointer += 1
    # Write the student data to that exact memory index
    memory_pool[top_pointer] = student_obj

def pop_student():
    global top_pointer
    # Read the data currently at the pointer
    student = memory_pool[top_pointer]
    # Move the pointer DOWN, effectively "deleting" access to the old data
    top_pointer -= 1
```

2. The Queue: Two Pointers Chasing Each Other

A Queue (First-In, First-Out) requires two pointers: a head where students are removed, and a tail where new students are added.

Time Complexity: $O(1)$ for Insert/Delete. **I/O Cost:** Low, but data physically crawls rightward across the array until it wraps around.

```
memory_pool = [None] * 100
head_ptr = 0    # Points to the front of the line
tail_ptr = 0    # Points to the next empty slot at the back

def enqueue_student(student_obj):
    global tail_ptr
    # Write the data to the tail index
    memory_pool[tail_ptr] = student_obj
    # Move tail right (modulo used to wrap around to index 0 if needed)
    tail_ptr = (tail_ptr + 1) % len(memory_pool)

def dequeue_student():
    global head_ptr
    # Read the data from the head index
    served_student = memory_pool[head_ptr]
    # Move head right, "forgetting" the served student
```

3. The Linked List: Explicit Node Pointers

Stacks and Queues require data to sit next to each other. A Linked List scatters data anywhere in RAM. We achieve this by storing small lists: [Student_Obj, Next_Index].

Time Complexity: $O(1)$ Insert, but a brutal $O(n)$ Search. **I/O Cost:** High read latency due to pointer jumping.

```
# Memory is completely fragmented! We use -1 to mean "Null" (End of list)
# Format: [Payload, Pointer_to_Next]
memory_pool = [
    [alice_obj, 2],    # Index 0: Alice points to Index 2
    [None, -1],       # Index 1: Empty junk memory
    [bob_obj, -1],    # Index 2: Bob points nowhere (End of line)
    [charlie_obj, 0]  # Index 3: Charlie points to Index 0
]

# The Head Pointer tells us where to start reading!
head_ptr = 3

# To traverse: Start at 3 (Charlie) -> jump to 0 (Alice) -> jump to 2 (Bob)
```

4. Binary Search Trees: Dual Pointers

To fix the $O(n)$ search time of a Linked List, we simply add a second pointer. Format: [Student_Obj, Left_Index, Right_Index].

Time Complexity: $O(\log n)$ Search if balanced. We divide the search space in half with every pointer hop!

```
# Format: [Payload, Left_Pooter, Right_Pooter]
# Left points to a lower GPA, Right points to a higher GPA
memory_pool = [
    [bob_obj (GPA 3.2),      1,  2], # Index 0: Root.
    [charlie_obj (GPA 2.8), -1, -1], # Index 1: Left Child (Lower)
    [alice_obj (GPA 3.8),   -1, -1]  # Index 2: Right Child (Higher)
]

root_ptr = 0

# If we search for GPA 3.8, we start at 0. Bob is 3.2.
# 3.8 is higher, so we follow the Right_Pooter to Index 2. Found in  $O(\log n)$ !
```

5. Heaps: Mathematical Pointers (CLRS Chapter 6)

A Heap (Priority Queue) is a Tree, but it is so perfectly balanced that we don't even need to store explicit left/right pointers. We use **math** on the indices!

Time Complexity: $O(\log n)$ Insert/Delete. **I/O Cost:** Zero wasted memory on pointers!

```
# We store just the student objects. No explicit pointers needed!
heap_memory = [student_99, student_85, student_92, student_70]

# Math dynamically generates the pointers!
def get_parent_index(i):
    return (i - 1) // 2

def get_left_child_index(i):
    return 2 * i + 1

def get_right_child_index(i):
    return 2 * i + 2

# Example: Who is the left child of student_85 (Index 1)?
# Left_Child = (2 * 1) + 1 = Index 3 (student_70).
```

The Grand Unification of Dynamic Sets

Structure	Pointer Mechanism	Worst-Case Search	Insert/Delete I/O
Flat Array	None (Contiguous)	$O(n)$	Heavy (Shifting $O(n)$)
Stack / Queue	Single Boundary Ptr	$O(n)$	Light ($O(1)$)
Linked List	Single Explicit Ptr	$O(n)$	Light ($O(1)$)
Binary Tree	Dual Explicit Ptrs	$O(n)$ (if unbalanced)	Light ($O(1)$)
Heap	Dual Math Ptrs	$O(n)$	Med (Swapping $O(\log n)$)
BST	Sorted Dual Ptrs	$O(\log n)$	Light ($O(1)$)

Conclusion: By manipulating how and where we store pointer indices, we completely control the Time and I/O efficiency of our software!